

TECHNICAL REFERENCE

AI Usage and Tools

AI Tools and Model Usage

Technology	Role
httpx (OpenAI-compatible POST /chat/completions + tools)	LLM chat transport for Inception and Groq; no vendor SDK dependency
Inception mercury-2.5	Preferred chat/tool-calling model
Groq openai/gpt-oss-20b	Fallback chat/tool-calling model
Groq Whisper whisper-large-v3-turbo	Optional STT provider
Browser Web Speech API (speechSynthesis)	Client-side TTS for chat/voice replies
Twilio Programmable Voice + Media Streams	Telephone transport (Twiml, HMAC-SHA1, audio streaming); not an AI model
Redis (careflow:ai-context:, TTL 7200 s)	Structured AIContext conversation memory; Celery broker/backend
Pydantic v2 tool schemas + JSON Schema	Function/tool specifications sent to the model
Custom in-process MCP layer (20 tools) + AgentToolClient	Controlled capability boundary (auth, idempotency, bounded retry, audit)
Custom energy-VAD over PCM16	Speech gating to avoid STT hallucination on silence
WebRTC / WebSocket audio (AudioWorklet, WAV playback queue)	Browser voice capture/playback and server voice loops
python-multipart	Form parsing for Twilio webhooks
pytest + Playwright	Application test and E2E verification used during AI-assisted development
Framer Motion, Tailwind, lucide-react	Frontend motion/design implementation (not AI models)

This document records the AI tools, AI models, AI APIs, and AI-assisted development practices used during the development of CareFlow AI, a multi-tenant healthcare appointment scheduling platform.

Contents

1. 1. Purpose
2. 2. AI Tools Used During Development
3. 3. AI Models / Providers Used in the Application
4. 4. AI Features Implemented in the Application
 - 4.1 Feature 1 — Conversational scheduling assistant (text chat)
 - 4.2 Feature 2 — Web voice conversation
 - 4.3 Feature 3 — Telephone conversation (Twilio)
 - 4.4 Feature 4 — Questionnaire collection with safety guard and concern-flag escalation
 - 4.5 Feature 5 — Human escalation (`transfer\_to\_human`)
5. 5. AI-Assisted Development Workflow
6. 6. AI Prompts / Prompt Engineering
 - 6.1 6.1 `SYSTEM\_PROMPT` (scheduling assistant)
 - 6.2 6.2 `TELEPHONY\_GUARD\_PROMPT` + `TELEPHONY\_OPEN\_TOOLS` + `\_telephony\_gate`
 - 6.3 6.3 `WEB\_VOICE\_PROMPT`
 - 6.4 6.4 Deterministic safety strings and classifiers (not LLM prompts, but prompt-adjacent logic)
 - 6.5 6.5 Tool descriptions as model instructions
 - 6.6 6.6 Figma design brief (development prompt, not runtime prompt)
7. 7. RAG / Knowledge Retrieval
8. 8. AI Agent / Tool Calling / MCP
9. 9. Voice / Audio AI
 - 9.1 Speech-to-Text
 - 9.2 Text-to-Speech
 - 9.3 Voice Orchestration
10. 10. AI-Assisted Testing and Debugging
11. 11. AI-Assisted UI / Design
12. 12. AI-Assisted Documentation
13. 13. Human Responsibility and Review
14. 14. Technologies Related to AI
15. 15. Summary

AI Usage and Tools

1. Purpose

This document records the AI tools, AI models, AI APIs, and AI-assisted development practices used during the development of CareFlow AI, a multi-tenant healthcare appointment scheduling platform.

It distinguishes between:

- AI tools used to **build/develop** the project (development-time assistance).
- AI models and providers used **inside the application** at runtime.
- AI-assisted design, documentation, testing, and debugging activities.

All claims are grounded in repository evidence. Where information cannot be verified from the repository, this is explicitly stated. No secrets, API keys, or private credentials are included.

2. AI Tools Used During Development

Tool	Category	How It Was Used	Evidence/Location
OpenCode	AI Coding Assistant	Phased implementation of backend, frontend, voice, workflow, observability, and tests. Each build phase was executed through an explicit implementation prompt recorded verbatim, with scope limited to one phase at a time, followed by tests and verification against acceptance criteria. This includes scaffolding, auth/RBAC, onboarding, configuration, scheduling, patients, Mock EHR, appointments, reliability, MCP server + agent, workflow/notifications, questionnaires, web voice, dashboards, telephone, observability, testing pass, frontend UX rebuilds, motion redo, and fix passes.	docs/opencode-prompts.md (entire file; e.g. Phase 0 scaffolding prompt, Phase 9 MCP Server + AI Agent prompt, Phase 12 Web Voice prompt, Phase 14 Telephone prompt, UI polish passes); build-plan-deep-dive.md (phase scope authority); architecture-design.md (stack authority); git history (56 commits on main, e.g. 19c9580 feat: add MCP capability tools and text scheduling agent, 98432a9 feat: add web voice loop with VAD, barge-in, and silence handling, b43f10c feat: add telephone channel with caller identity verification)
Figma Make / Figma AI	AI Design Tool	A Figma design brief was authored in the repository for use in Figma or Figma Make / Figma AI. The brief specifies product overview, design system, all patient / hospital-admin / doctor screens, global patterns, and deliverables. Whether the brief was actually executed inside Figma AI / Figma Make is not verifiable from the repository .	prompt.txt:1-5 ("Paste this whole file into Figma (or Figma Make / Figma AI) as the design brief") through prompt.txt:151-159 (deliverables)

The following tools were explicitly searched for and **not found** in the repository, and are therefore **not claimed**: Claude, Claude Code, GPT, OpenAI product, Gemini / Google AI, Groq console/tooling as a dev assistant, GLM, Z.AI, Ollama, Qwen, GitHub Copilot, Cursor, Antigravity, ChatGPT, or any other AI coding assistant. The absence of

evidence was checked via repository-wide content search, backend/requirements.txt, frontend package.json files, docs/, scripts/, infra/, and git history. Commit messages contain no references to these tools.

3. AI Models / Providers Used in the Application

Provider / Model	Purpose	Where Used	Evidence/Location
Inception Labs — mercury-2.5 (OpenAI-compatible chat completions + function/tool calling)	Text generation; agent orchestration via tool calling for scheduling conversations	Preferred chat backend in groq_complete() / _chat_backend(); used by text chat and voice agent turns	backend/app/ai/agent/orchestrator.py:658-730 (_chat_backend, groq_complete); backend/app/core/config.py:48-52 (INCEPTION_API_KEY, INCEPTION_MODEL default mercury-2.5, INCEPTION_BASE_URL=https://api.inceptionlabs.ai/v1); infra/.env.example:20-24; backend/.env.example:18; Architecture.md:41-42,92
Groq — openai/gpt-oss-20b (OpenAI-compatible chat completions + function/tool calling)	Text generation; fallback chat model when Inception key is absent; also referenced for rate-limit retry handling	Fallback branch of _chat_backend(); same POST {base}/chat/completions path with tools=[{type:function}], tool_choice=auto, 4-attempt retry on 429/5xx	backend/app/ai/agent/orchestrator.py:658-730; backend/app/core/config.py:39-45 (LLM_API_KEY with GROQ_API_KEY alias, LLM_MODEL default openai/gpt-oss-20b, LLM_BASE_URL=https://api.groq.com/openai/v1); infra/.env.example:16-19; docs/opencode-prompts.md:289-296 (default model changed from retired llama-3.3-70b-versatile to openai/gpt-oss-20b); Architecture.md:41-42
Groq Whisper — whisper-large-v3-turbo	Speech-to-Text (STT)	Optional STT provider (GroqSTT); wraps 16-bit PCM mono 16 kHz audio as in-memory WAV and posts to {LLM_BASE_URL}/audio/transcriptions with temperature=0, language=en, 3-attempt retry on 429/5xx/timeout	backend/app/voice/stt_provider.py:1-139 (esp. :64-110); backend/app/core/config.py:60-64 (STT_PROVIDER default stub, STT_MODEL default whisper-large-v3-turbo); infra/.env.example:43-44; Architecture.md:93

Provider / Model	Purpose	Where Used	Evidence/Location
Browser Web Speech API (<code>window.speechSynthesis</code> + <code>SpeechSynthesisUtterance</code>)	Text-to-Speech (TTS) for chat and voice replies; client-side only, no network call, no external TTS API	Patient frontend hook used by chat and voice pages; <code>markdown/citations/code</code> stripped before speaking; <code>speak</code> (single reply), <code>enqueue</code> (streamed voice sentences), <code>stop</code>	<code>frontend/apps/patient/src/voice/useSpeechSynthesis.ts:1-135</code> ; <code>backend/app/voice/tts_provider.py:1-9</code> (confirms browser path); <code>commit fa14263</code> ("browser-native <code>SpeechSynthesis</code> for chat and voice, drop server <code>GroqTTS</code> "); <code>Architecture.md:93</code>
Groq Orpheus — <code>canopylabs/orpheus-v1-english</code> (voice troy)	Former server-side TTS provider — removed, not active	Previously targeted server TTS endpoint; removed because the model required additional organization terms acceptance (verified 400) and is now absent from code. <code>TTS_MODEL</code> / <code>TTS_VOICE</code> keys remain as configuration defaults only. The active server TTS interface is <code>StubTTS</code> (raises on use); <code>chat/voice</code> speech uses the browser API above.	<code>backend/app/voice/tts_provider.py:1-51</code> ("external Groq Orpheus implementation has been removed"); <code>backend/app/core/config.py:65-69</code> ; <code>infra/.env.example:45-47</code> ; <code>docs/opencode-prompts.md:419-424,607-623</code> (TTS decommissioning history); <code>Architecture.md:93</code> ("server Orpheus removed")

Not used by the application: OpenAI API/models, Anthropic/Claude models, Google/Gemini models, Qwen, GLM/Z.AI, Ollama, embedding models, vector databases, LangChain, LangGraph, or any MCP protocol SDK. `backend/requirements.txt` contains no `openai`, `anthropic`, `google-generativeai`, `langchain`, `langgraph`, `chromadb`, `pinecone`, `faiss`, `sentence-transformers`, or `mcp` package; LLM access is via `httpx` to OpenAI-compatible endpoints only.

4. AI Features Implemented in the Application

Feature 1 — Conversational scheduling assistant (text chat)

- Purpose: Help signed-in patients find hospitals/doctors, check real availability, and book, reschedule, or cancel visits conversationally, plus collect pre-visit questionnaires.
- User interaction: Patient sends text via `POST /chat {message, conversation_id, latitude/longitude, selection?}`; receives `ChatOut {reply, doctors, slots, appointment_types, day_schedule, consultation_modes, pending_booking, booking_stage, hospital/selected_date/selected_start/selected_end/duration_minutes/missing_fields}` rendered as cards, slot grids, type/mode selectors, date strips, and confirm panels. Widget taps re-enter as typed `booking_selection` events plus transcript text.
- AI component/model: Inception `mercury-2.5` preferred, Groq `openai/gpt-oss-20b` fallback; hand-rolled tool-calling loop (`MAX_ITERATIONS=8`); deterministic guards and extractors; Redis-backed structured `AIContext`.
- Input: Free text plus optional `conversation_id`, geolocation, and typed `selection {field, value}`.
- Processing: Deterministic pre-LLM extractors and gates, system prompt + serialized context + last 10 history messages sent to chat-completions with function specs, tool calls executed via `AgentToolClient`, context updated after each tool result, response assembled with structured UI payload.

- Output: Short conversational reply plus structured booking UI state; bookings only after explicit confirmation; confirmed vs parked vs failed outcomes relayed honestly.
- Relevant implementation files: `backend/app/ai/router.py` (ChatIn, ChatOut, POST /chat); `backend/app/ai/agent/orchestrator.py` (SYSTEM_PROMPT, run_conversation, gates, extractors, _chat_backend, groq_complete); `backend/app/ai/agent/booking_state.py`; `backend/app/ai/context/ai_context.py`; `backend/app/ai/mcp_client/client.py`; `frontend/apps/patient/src/api.ts` (postChat); `frontend/apps/patient/src/pages/ChatPage.tsx`; `frontend/apps/patient/src/components/ai/ai.tsx`.

Feature 2 — Web voice conversation

- Purpose: Complete the same scheduling flow by spoken voice in the browser.
- User interaction: Microphone capture with Voice Activity Detection, live transcript bubbles, interrupt/hang-up controls, spoken replies; WS /voice/ws JSON frames (audio/partial/final/agent_text/audio_out/interrupt/state/ended).
- AI component/model: Same orchestrator (run_conversation(channel="web_voice") with per-turn WEB_VOICE_PROMPT); STT via Stub (default) or Groq Whisper; spoken output via browser Web Speech API; server stubTTS interface retained only for the WS pipeline.
- Input: 16-bit PCM mono audio chunks (browser AudioWorklet) plus conversation_id / resume parameter.
- Processing: Energy-VAD gating, partials display-only (never trigger writes), full-turn STT → orchestrator → per-sentence spoken output; barge-in cancels both LLM loop (should_stop) and TTS stream; single silence re-prompt then human escalation.
- Output: Spoken short sentences plus transcript/agent-text frames; structured cards are not delivered in-call (chat-only widgets).
- Relevant implementation files: `backend/app/voice/web_voice/ws_handler.py`; `backend/app/voice/session_manager.py`; `backend/app/voice/vad.py`; `backend/app/voice/stt_provider.py`; `backend/app/voice/tts_provider.py`; `backend/app/voice/router.py`; `frontend/apps/patient/src/voice/useWebRTCAudio.ts`; `frontend/apps/patient/src/voice/useSpeechSynthesis.ts`.

Feature 3 — Telephone conversation (Twilio)

- Purpose: Handle inbound phone calls with the same agent pipeline, with strict caller-identity verification before any patient-data access.
- User interaction: Caller dials Twilio number; webhook returns TwiML <Connect><Stream>; audio flows over Twilio Media Streams WebSocket; agent asks for full name + date of birth, verifies, then proceeds with scheduling.
- AI component/model: Same orchestrator with TELEPHONY_GUARD_PROMPT and deterministic telephony gate; verify_caller_identity tool (3 attempts, then transfer_to_human); STT same as web voice; TTS via server stub/browser path per pipeline.
- Input: G.711 mu-law telephony audio (8 kHz ↔ 16 kHz resampled), caller phone number, spoken name/DOB.
- Processing: Phone lookup (exact + 10-digit suffix, normalized name/DOB check that never reveals stored values); sessions start UNVERIFIED; allowlist of open tools (verify_caller_identity, transfer_to_human, search_hospitals, search_doctors, check_availability, get_context); verification flag refreshed mid-turn; call drops treated as standard unknown-outcome cases.
- Output: Spoken replies over media stream; verified bookings follow the same confirm-gate and verify/sync/reconcile path as chat.
- Relevant implementation files: `backend/app/voice/telephony/twilio_webhook.py`; `backend/app/voice/telephony/media_stream_handler.py`; `backend/app/voice/telephony/identity.py`; `backend/app/voice/telephony/audio.py`; `backend/app/mcp_server/tools/verify_caller_identity.py`; `backend/app/ai/agent/orchestrator.py`:265-330 (telephony prompt, allowlist, gate).

Feature 4 — Questionnaire collection with safety guard and concern-flag escalation

- Purpose: Collect only the configured pre-visit questionnaire answers conversationally, without inventing clinical questions or interpreting answers medically.
- User interaction: Agent asks the fetched questions in order using exact prompts; patient answers in chat; completion gated on required answers; concerning free-text triggers an existing human escalation notice.
- AI component/model: Same chat model constrained by system-prompt section, full tool docstrings shipped to the model, and `completed` / `missing_required` gating in tool results; keyword flag rule (`FLAG_PHRASES`) opens an Escalation with no medical interpretation.
- Input: `get_questionnaire` / `submit_questionnaire` tool I/O; appointment-scoped question sets.
- Processing: Scope precedence (doctor > type > specialty > hospital); hard 422 errors vs draft (`completed=false`) for missing required; deterministic agent guard in three places.
- Output: Structured `QuestionnaireResponse` rows; doctor read-only view; escalation row when flagged.
- Relevant implementation files: `backend/app/domain/questionnaire/` (`models.py`, `service.py`, `router.py`); `backend/app/mcp_server/tools/get_questionnaire.py`; `backend/app/mcp_server/tools/submit_questionnaire.py`; `backend/app/ai/agent/orchestrator.py` (questionnaire prompt section); `backend/tests/test_questionnaire.py`.

Feature 5 — Human escalation (`transfer_to_human`)

- Purpose: Hand off to a human operator when the request is clinical, ambiguous, silent, unverified, or failing.
- User interaction: Agent reply offers connection; frontend shows escalation notice; operator resolves via reconciliation/operations views.
- AI component/model: Same chat model invoking the `transfer_to_human` capability; deterministic triggers (clinical pre-check, loop exhaustion, silence, identity lockout, flagged questionnaire).
- Input: `conversation_id`, `reason`.
- Processing: Persists an Escalation row; audited as a capability execution; surfaced in dashboards.
- Output: `escalated=true` in chat response; escalation record for operators.
- Relevant implementation files: `backend/app/mcp_server/tools/transfer_to_human.py`; `backend/app/mcp_server/escalations.py`; `backend/app/ai/agent/orchestrator.py` (clinical decline, `LOOP_EXHAUSTED`, silence path).

5. AI-Assisted Development Workflow

Repository-confirmed usage (via `docs/opencode-prompts.md`, phase plan, and git history):

- Code generation: backend skeleton, domain modules, MCP tools, orchestrator, voice pipeline, Celery workflow, routers, migrations, and frontend apps/pages/components were implemented incrementally through OpenCode phase prompts. Each phase section records the exact prompt, scope implemented, files created/modified, and tests added/run.
- Test generation: pytest suites and Playwright E2E specs were added per phase (e.g. Phase 9 live smoke, Phase 10 workflow tests, Phase 11 questionnaire tests, Phase 12 voice tests, Phase 14 telephony tests, Phase 16 categorized unit/ + integration/ + ai/ + ehr/ + e2e/ pass). `docs/opencode-prompts.md:659-690` records 43 new tests plus two Playwright specs for Phase 16.
- Debugging and fix passes: cross-phase audits and “fix all” sweeps are documented (CORS origin fix, `.dockerignore` secret-baking fix, model default fix, 422 mapping, dead-import removal, orphaned voice-task cancellation, reschedule/cancel round-trip coverage). Verification included full pytest runs, migration-head checks, compose validation, frontend `tsc + vite build`, and live Postgres smoke tests.

- Refactoring and UI iteration: documented rebuilds include directory endpoints for the UI, design-system v2, Framer Motion redo, icon replacement (emoji → stroke SVGs), Inter typography, slot day-column local-day fix, port allocation, and chat humanization/location-aware booking.
- Prompt engineering: system prompts, telephony/voice prompts, tool docstrings as model instructions, confirmation/completeness gates, and deterministic extractors were iterated in code (see §6).
- API integration: LLM chat-completions integration, Whisper STT integration, Twilio webhook/media-stream integration, and Mock EHR fault-injection for recovery demos were implemented and smoke-tested per phase notes.
- Architecture discussions: `architecture-design.md`, `Architecture.md`, `build-plan-deep-dive.md`, and `docs/appointment-ai-current-state.md` (read-only audit stating no LangGraph exists) guided and constrained implementation.
- Error analysis: live failure findings are recorded (e.g. retired Groq model, decommissioned TTS model needing terms acceptance, baked `.env` secret, trailing-space shell issue, port conflicts).

Not verifiable from the repository (intentionally not claimed):

- Use of AI chat assistants for explanations, research, or architecture Q&A outside the recorded OpenCode prompts.
- Use of AI debugging assistants, inline IDE completions, or AI code-review tools.
- Time saved, productivity effects, or developer-experience claims.
- Any AI tool involvement in deployment operations beyond the code and docs present.

6. AI Prompts / Prompt Engineering

6.1 `SYSTEM_PROMPT` (scheduling assistant)

- Purpose: Define the assistant as a scheduling-only care coordinator; enforce warmth/brevity, one-step-at-a-time booking (doctor → day → type → mode → time → confirm), IST handling, tool-grounded facts, no invented IDs/times/bookings, parked/failed outcome handling, location-aware search, greeting/small-talk behavior, questionnaire discipline, and confirm-first booking discipline.
- Location: `backend/app/ai/agent/orchestrator.py:42-214`.
- Input: System message plus serialized `AIContext` JSON, patient profile/location snippet, current date (IST), last 10 history messages, and generated tool specs.
- Expected output: Either a short conversational reply or OpenAI-compatible `tool_calls` following the booking state machine.
- How used: Prepended to every `groq_complete()` call in `run_conversation()` (`orchestrator.py:1758` plus channel appends); supplemented by deterministic gates so the prompt alone is not the safety boundary.

6.2 `TELEPHONY_GUARD_PROMPT` + `TELEPHONY_OPEN_TOOLS` + `_telephony_gate`

- Purpose: Require spoken full-name + DOB verification via `verify_caller_identity` before any patient-data tool on telephone calls; limit unverified calls to public reads plus verify/transfer; escalate after three failures.
- Location: `backend/app/ai/agent/orchestrator.py:265-330`.
- Input: Telephone channel flag, verification state, requested tool name.
- Expected output: Model asks for identity then calls verify; deterministic gate returns `caller_identity_required` for disallowed tools until verified.

- How used: Appended to system prompt for telephony turns (`orchestrator.py:1774`); enforced in code before every tool call.

6.3 WEB_VOICE_PROMPT

- Purpose: Tell the model the turn is live voice heard aloud (not text chat); require short spoken sentences without markdown/lists/IDs.
- Location: `backend/app/ai/agent/orchestrator.py:303-309`, wired at `orchestrator.py:1778`.
- Input: `channel="web_voice"` for the current turn.
- Expected output: Concise speakable reply text.
- How used: Per-turn append only; never persisted to `AIContext`.

6.4 Deterministic safety strings and classifiers (not LLM prompts, but prompt-adjacent logic)

- Purpose: Guarantee safety even if the model is confused: keyword clinical pre-check, greeting-only replies, visit-type/completeness/confirmation gates, decline/escalation messages.
- Location: `backend/app/ai/agent/orchestrator.py:216-263` (`MAX_ITERATIONS=8`, `CLINICAL_PATTERNS`, `CLINICAL_DECLINE`, `LOOP_EXHAUSTED`, `STOPPED`), gates at `orchestrator.py:267-508,918-997`, extractors at `orchestrator.py:807-1109,1398-1423`; canonical extractors/readiness/invalidation in `backend/app/ai/agent/booking_state.py`.
- Input: Raw user text and structured context/tool results.
- Expected output: Decline, greeting, gate refusal (`date_required`, `type_required`, `mode_required`, mismatch codes), or validated state update.
- How used: Executed in Python before/around LLM calls; results shape the messages sent to the model and the UI snapshot returned.

6.5 Tool descriptions as model instructions

- Purpose: Constrain tool use (e.g. questionnaire guard: ask only fetched questions in order; booking tools require explicit confirmation and idempotency keys).
- Location: Per-tool docstrings under `backend/app/mcp_server/tools/*.py`, shipped in full via `AgentToolClient.specs()` (`backend/app/ai/mcp_client/client.py:24-36`) and registry metadata (`backend/app/mcp_server/tools/_base.py:97-107`).
- Input: Tool input schemas (Pydantic models) and descriptions.
- Expected output: Valid function arguments conforming to JSON schemas.
- How used: Converted to OpenAI `tools=[{type:function}]` payload in `groq_complete()`.

6.6 Figma design brief (development prompt, not runtime prompt)

- Purpose: Instruct a design tool to produce tokens, components, screens, and prototypes for all apps.
- Location: `prompt.txt` (entire file).
- Input: Product overview, design system, screen specifications.
- Expected output: Figma file with three sections plus clickable prototypes (per `prompt.txt:151-159`).
- How used: Intended to be pasted into Figma / Figma Make / Figma AI; actual execution not verifiable (see §2, §11).

No prompt files contain secrets. Model keys are referenced only by environment-variable name.

7. RAG / Knowledge Retrieval

RAG was **not found** and is explicitly documented as not used.

- No document ingestion, text extraction, chunking, embedding generation, vector storage, similarity search, retrieval, context construction from documents, or citation/grounding over documents exists in the codebase.
- Repository-wide search for `langgraph|langchain|chromadb|pinecone|faiss|weaviate|qdrant|pgvector|embedding|vector_store|sentence-transformer` returns no hits in backend/.
- `Architecture.md:12,92,329` states: “AI: Tool-calling agent (Inception preferred, Groq fallback) — **not RAG**” and “No embeddings, chunking, vector DB, or retrieval exist in the codebase. The assistant is a deterministic-guarded tool-calling loop over an OpenAI-compatible chat API.”
- `docs/appointment-ai-current-state.md §A, §C` confirms scheduling facts come from live database/EHR tools, not documents.

Instead of RAG, the application uses:

- Live capability tools over PostgreSQL and the Mock EHR connector (`backend/app/mcp_server/tools/*.py` → domain services).
 - Structured conversation memory (AIContext in Redis, TTL 7200 s) rather than retrieved passages.
 - Deterministic extractors and readiness evaluation (`backend/app/ai/agent/booking_state.py`).
-

8. AI Agent / Tool Calling / MCP

Custom in-process capability layer is implemented. No LangGraph, LangChain, MCP protocol SDK, or external agent framework is used.

- Agent/orchestrator: hand-rolled OpenAI-compatible tool loop in `backend/app/ai/agent/orchestrator.py:1352` (`run_conversation`), `MAX_ITERATIONS=8` (`orchestrator.py:216`), with `groq_complete()` (`orchestrator.py:680-730`) posting `{model, messages, tools, tool_choice:auto}` to `{base}/chat/completions` via `httpx`, 4-attempt retry on 429/5xx with vendor-header-aware backoff (`_throttle_delay_s`).
 - Available tools (20, asserted in code): `search_hospitals, search_doctors, check_availability, get_day_schedule, lookup_patient, get_appointment, create_appointment, reschedule_appointment, cancel_appointment, get_questionnaire, list_appointment_types, submit_questionnaire, send_notification, start_workflow, get_context, update_preferences, verify_caller_identity, verify_external_appointment, synchronize_state, transfer_to_human`. Registry and HTTP surface: `backend/app/mcp_server/server.py:55-83` plus `GET /mcp/tools`, `POST /mcp/call`.
 - Tool invocation flow: `orchestrator` → `AgentToolClient.call(name, arguments)` (`backend/app/ai/mcp_client/client.py:38-52`, faults returned as `{ok:false, error}` so turns never crash) → `server.execute_tool()` (Pydantic validation → 422 on invalid input) → `mcp_tool` wrapper (`backend/app/mcp_server/tools/_base.py:54-95`: `auth` → `idempotency` → `bounded retry for reads` → `audit`) → domain service → PostgreSQL / EHR connector.
 - State/context management: AIContext Pydantic model (`backend/app/ai/context/ai_context.py`, 37 fields after Phase 2) stored in Redis `careflow:ai-context:{conversation_id}` (TTL 7200 s, in-memory fallback); loaded at turn open, mutated by deterministic extractors and `_apply_result_to_context` / `_remember_booking_selection`, saved during and at end of turn; history capped at 20 turns, last 10 sent to the model. Conversation identifier is client-generated (`conversation_id`; voice resume/telephone custom parameter).
-

- Model interaction: system prompt + context JSON + history + tool specs → model → zero or more tool calls per iteration → gates (`_telephony_gate`, `visit_type_gate`, `booking_completeness`, `_booking_confirmation_gate`) → execution → context update → loop or reply assembly (ChatOut with structured UI fields).
- Relevant files: `backend/app/ai/agent/orchestrator.py`; `backend/app/ai/agent/booking_state.py`; `backend/app/ai/context/ai_context.py`; `backend/app/ai/mcp_client/client.py`; `backend/app/mcp_server/server.py`; `backend/app/mcp_server/tools/_base.py`; `backend/app/mcp_server/middleware/*`; `backend/app/ai/router.py`.

Framework non-use is verified: `docs/appointment-ai-current-state.md:5-10` ("there is no LangGraph in this repository"); `Architecture.md:364` ("MCP is custom in-process code, not the MCP protocol SDK"); `backend/requirements.txt` has no agent-framework dependency.

9. Voice / Audio AI

Speech-to-Text

- Technology/provider: Stub provider by default (hears nothing; tests inject scripted providers); optional Groq Whisper (`whisper-large-v3-turbo`) when `STT_PROVIDER=groq` and a key is configured.
- Input: 16-bit PCM mono 16 kHz audio (web) or resampled telephony audio (8 kHz mu-law ↔ 16 kHz PCM); sub-200 ms buffers short-circuit to empty without a vendor call.
- Processing: In-memory PCM→WAV packaging, `POST {LLM_BASE_URL}/audio/transcriptions` (`temperature=0`, `language=en`, 60 s timeout), 3-attempt retry on timeout/429/5xx; VAD gating and partial-vs-final discipline in WS handlers.
- Output: `STTResult {text, language}` consumed by the agent loop.
- Relevant files: `backend/app/voice/stt_provider.py`; `backend/app/voice/vad.py`; `backend/app/voice/web_voice/ws_handler.py`; `backend/app/voice/telephony/media_stream_handler.py`; `backend/app/voice/telephony/audio.py`; `backend/app/core/config.py:60-64`.

Text-to-Speech

- Technology/provider: Browser Web Speech API for chat and voice replies (no server request, no external key). Server-side external TTS (Groq Orpheus `canopylabs/orpheus-v1-english`) was **removed**; only StubTTS (raises `TTSError`) plus test hooks remain for the WS/telephone pipeline interface.
- Input: Agent reply sentences / streamed `agent_text`.
- Processing: Client cleans markdown/citations/code/URLs via `cleanTextForSpeech()` then `speak/enqueue`; server `_speak` streams sentences as text with best-effort audio rather than truncating on TTS failure; interrupt/hang-up/unmount cancels speech.
- Output: Audible speech in the browser; TTS-failed frames are benign/ignored where applicable.
- Relevant files: `frontend/apps/patient/src/voice/useSpeechSynthesis.ts`; `backend/app/voice/tts_provider.py`; `frontend/apps/patient/src/pages/ChatPage.tsx`; `frontend/apps/patient/src/pages/VoiceChat.tsx`; `frontend/apps/patient/src/pages/VoicePage.tsx`; `commit fa14263`.

Voice Orchestration

Web and telephone channels share one `run_conversation()` orchestrator and MCP client; voice-specific code lives only at transport edges. Web voice passes `channel="web_voice"` per turn (short spoken replies); telephony uses `channel="telephony"` with identity allowlisting. Barge-in aborts both generation (`should_stop`) and speech; silence re-prompts once then escalates. Structured booking widgets are chat-only; voice turns receive prose. Telephony

transport uses Twilio Programmable Voice TwiML <Connect><Stream>, HMAC-SHA1 validation when TWILIO_AUTH_TOKEN is set, and 503 fail-closed behavior when TELEPHONY_STREAM_URL is empty. Relevant files: backend/app/voice/router.py; backend/app/voice/session_manager.py; backend/app/voice/web_voice/ws_handler.py; backend/app/voice/telephony/media_stream_handler.py; backend/app/voice/telephony/twilio_webhook.py.

10. AI-Assisted Testing and Debugging

Application tests present (not themselves AI models):

- Backend: 30+ pytest files including backend/tests/test_booking_state.py (45 deterministic booking-state tests), backend/tests/test_voice.py, backend/tests/test_telephony.py (22 tests), backend/tests/test_mcp_agent.py, backend/tests/ai/ (intent, context, clarification, tool selection, safety boundary), backend/tests/ehr/, backend/tests/integration/, backend/tests/unit/, plus domain suites (auth, scheduling, appointments, reliability, workflow, questionnaire, dashboards, observability). Suite sizes are recorded per phase in docs/opencode-prompts.md (e.g. 154/154, 165/165, 174/174, 185/185, 209/209, 220, 263/263, 376+).
- E2E: Playwright + Chromium headless specs happy_path_spec and failure_recovery_spec in backend/tests/e2e/ with own package.json (@playwright/test).
- Config coverage: backend/tests/test_health.py:57-69 asserts documented settings keys including LLM/STT/TTS/Twilio keys.

AI-assisted development activities (repository-confirmed):

- Test generation and test debugging were performed through the OpenCode phased workflow: each phase added tests, ran the suite, fixed failures, and recorded results in docs/opencode-prompts.md. Examples include narrowing Phase 9 stubs, adding reschedule/cancel/synchronize round-trip coverage, and replacing the obsolete GroqTTS retry test with a stub-interface test.
- API debugging via live smoke tests on scratch Postgres/Redis stacks is recorded per phase (tool registry listing, slot retrieval, parked-booking resolution, clinical decline, keyless 503, fault-mode timeout/recovery, TwiML match, trace/metrics checks).
- Frontend debugging via tsc + vite build, bundle integrity checks, preview serving, and headless Playwright reruns is recorded (including port-conflict and CORS fixes).
- Backend debugging via migration-head checks, compose validation, worker-log inspection, and fault-injection scenarios is recorded.

Clearly separated and **not claimed**: standalone AI test generators, AI failure-analysis bots, or AI deployment troubleshooters. No such tool is evidenced in the repository; troubleshooting evidence consists of the OpenCode fix-pass notes above.

11. AI-Assisted UI / Design

Tool	Purpose	What was generated/designed	How incorporated
Figma Make / Figma AI (brief prepared)	Produce design tokens, component library, all patient / hospital-admin / doctor screens with loading/empty/error variants, and clickable prototypes	Brief covers colors, typography, shape, components, motion, responsive and accessibility patterns, and realtime annotations	Not verifiable from the repository whether Figma AI was executed or its output imported. The file present is the input brief only (prompt.txt). No Figma file, export, or import mapping is present.
OpenCode (phased frontend implementation)	Implement the working UI in code	Patient / doctor / hospital-admin / operations-admin / platform-admin React + Vite + Tailwind apps; hero search, doctor cards, 3-step booking wizard, visits/inbox/chat/voice/preferences/profile, admin overview/AI-activity/integration/analytics/operations, doctor agenda/calendar; design-system CSS, Inter typography, stroke icon set, Framer Motion language, slot-grid and timeline components	Directly committed code under frontend/apps/*; verified by tsc + vite build and Playwright specs per docs/opencode-prompts.md (UX rebuild, professional UI pass, Framer Motion redo, icon/motion polish)

No claim is made that Figma AI generated the committed UI. The committed UI is evidenced as code iterations through OpenCode phases.

12. AI-Assisted Documentation

- docs/opencode-prompts.md: verbatim record of OpenCode implementation prompts plus per-phase execution notes, maintained as part of the OpenCode workflow itself (the Phase 0 prompt explicitly requires creating/updating this file). This is the primary evidence of AI-assisted development.
- Architecture.md and submission-docs/Architecture.md: implementation-grounded architecture reference verified against backend/frontend/infra files; states its own verification method in its header.
- docs/appointment-ai-current-state.md: read-only architecture audit (dated 2026-09-20) tracing execution paths with file/line citations; explicitly states no code was modified for the report and that no LangGraph exists.
- architecture-design.md and build-plan-deep-dive.md: upstream design and phase plan used as authorities by the OpenCode prompts.
- README.md: placeholder only (# CareFlow-AI); not substantive documentation.
- Whether any prose in the above documents was drafted with AI chat assistance versus written directly by the developer is **not verifiable from the repository** and is therefore not claimed. No AI documentation-generator tool is evidenced.

13. Human Responsibility and Review

AI tools were used as development assistance only. Implementation, integration, testing, validation, and final decisions were reviewed by the developer, as evidenced by per-phase acceptance checks, full test-suite runs, migration and compose validation, frontend builds, live smoke tests against scratch stacks, fix-pass audits, and conventional commits on `main`. Safety-critical behavior (scheduling-only boundary, confirmation gates, identity verification, idempotent writes, verify-before-confirm, reconciliation escalation) is enforced in deterministic server code, not delegated to model output alone.

14. Technologies Related to AI

Technology	Role
httpx (OpenAI-compatible POST /chat/completions + tools)	LLM chat transport for Inception and Groq; no vendor SDK dependency
Inception mercury-2.5	Preferred chat/tool-calling model
Groq openai/gpt-oss-20b	Fallback chat/tool-calling model
Groq Whisper whisper-large-v3-turbo	Optional STT provider
Browser Web Speech API (speechSynthesis)	Client-side TTS for chat/voice replies
Twilio Programmable Voice + Media Streams	Telephone transport (Twiml, HMAC-SHA1, audio streaming); not an AI model
Redis (careflow:ai-context:, TTL 7200 s)	Structured AIContext conversation memory; Celery broker/backend
Pydantic v2 tool schemas + JSON Schema	Function/tool specifications sent to the model
Custom in-process MCP layer (20 tools) + AgentToolClient	Controlled capability boundary (auth, idempotency, bounded retry, audit)
Custom energy-VAD over PCM16	Speech gating to avoid STT hallucination on silence
WebRTC / WebSocket audio (AudioWorklet, WAV playback queue)	Browser voice capture/playback and server voice loops
python-multipart	Form parsing for Twilio webhooks
pytest + Playwright	Application test and E2E verification used during AI-assisted development
Framer Motion, Tailwind, lucide-react	Frontend motion/design implementation (not AI models)

Excluded: embeddings, vector databases, RAG pipelines, LangGraph/LangChain, MCP SDK, server-side Orpheus TTS (removed), and any undisclosed model API.

15. Summary

- AI development tools: OpenCode is the only repository-evidenced AI development tool, used for phased code, test, fix, UI, and docs workflows (`docs/opencode-prompts.md`, phase plan, git history). A Figma AI / Figma Make design brief was authored (`prompt.txt`), but its execution is not verifiable. No other AI assistant (Copilot, Cursor, ChatGPT, Claude, Gemini, etc.) is evidenced.
- AI providers/models used by the application: Inception `mercury-2.5` (preferred) and Groq `openai/gpt-oss-20b` (fallback) for tool-calling chat; Groq Whisper `whisper-large-v3-turbo` as optional STT; browser Web Speech API as the active TTS path. Server Groq Orpheus TTS was removed. All access is via `httpx` to OpenAI-compatible endpoints; keys are environment-only and `gitignore`d.
- AI-powered application features: tool-calling scheduling assistant (text), web voice conversation, telephone conversation with identity verification, questionnaire collection with concern-flag escalation, and human escalation — all mediated by 20 audited MCP-style tools over live PostgreSQL/EHR state, with Redis conversation memory and deterministic safety gates.
- AI-assisted development activities: OpenCode-driven code/test generation, cross-phase debugging and fix sweeps, prompt engineering (system/telephony/voice prompts, tool docstrings, deterministic extractors/gates), API/voice/EHR integration testing, frontend rebuilds, and phase documentation. RAG, embeddings, vector search, autonomous agents beyond the bounded tool loop, and AI-generated Figma output are explicitly not claimed.